Express y ApiNews

Guía para entender y demostrar todo el curso

Una aplicación web para practicar Node.js, Express, rutas, middleware, MySQL, validaciones y JWT.

Ejemplos reales del proyecto que realizamos. Puedes demostrar las actividades desde el navegador.

Curso completo con laboratorio web y MySQL real

Qué incluye el trabajo

El laboratorio reúne las prácticas iniciales y una API de noticias completa. Los primeros bloques enseñan módulos, eventos, funciones y rutas. ApiNews añade persistencia en MySQL, relaciones entre tablas y operaciones protegidas.

La página web es el cliente de la API. Al presionar un botón, envía una solicitud al servidor Express. No calcula ni inventa la respuesta: el servidor ejecuta la operación y devuelve JSON.

El proyecto incluye instrucciones, documentación Swagger, una colección Postman, un esquema MySQL y pruebas. El menú permite presentar cada actividad por separado.

Node.js y Express cumplen funciones distintas

JavaScript es el lenguaje. Node.js permite ejecutarlo fuera del navegador y acceder a funciones del servidor, como archivos, eventos y conexiones. Express es una biblioteca de Node para organizar un servidor web.

En nuestro proyecto, Node ejecuta app.js y Express recibe solicitudes HTTP. Cuando una ruta necesita información, espera una consulta de Sequelize a MySQL y después responde al navegador.

El bucle de eventos coordina trabajo asíncrono. Esperar una consulta con await no significa que toda la aplicación se detenga. Pero un cálculo largo y síncrono sí puede ocupar el hilo de JavaScript.

La práctica de módulos

operations.js contiene sumar, restar, multiplicar y dividir. module.exports permite que otro archivo use esas funciones. app.js las carga con require y el laboratorio llama a la misma lógica desde una ruta.

Al enviar 12 y 8, la respuesta devuelve suma 20, resta 4, multiplicación 96 y división 1.5. Si el divisor es cero, la demostración lo explica en lugar de mostrar un resultado inválido.

Aprende esta relación: exportar ofrece una función; importar permite usarla. La función sigue estando definida en un solo lugar y puede reutilizarse en varias prácticas.

En la web → Módulos y eventos → Calcular con el módulo

La práctica de eventos

EventEmitter crea un emisor. Con on registras una función que escucha un evento; con emit avisas que ese evento ocurrió y puedes enviar datos. En la práctica, el evento saludo recibe el nombre escrito en el formulario.

La respuesta enumera el recorrido: crear el emisor, registrar el listener, emitir el evento, ejecutar el saludo y continuar. Así puedes explicar qué sucede sin mostrar una terminal.

Este listener se ejecuta de forma síncrona cuando llamamos emit. El manejo de eventos no convierte automáticamente cualquier código en asíncrono; son conceptos relacionados, pero diferentes.

Prueba con tu nombre y revisa el orden de los cinco pasos

El primer servidor y sus rutas

MiPrimerExpress define una aplicación que recibe solicitudes y responde. Una ruta combina un método HTTP con un camino. GET / consulta el inicio; GET /home muestra otra respuesta; GET /error activa el ejemplo de error.

req representa lo que llega: parámetros, filtros, encabezados y cuerpo. res permite elegir el código de estado, añadir encabezados y enviar la respuesta. next pasa el control al siguiente middleware.

El laboratorio integra estas prácticas en rutas /api/practicas. El botón Primer servidor muestra un mensaje JSON y el encabezado que añadió el middleware.

La URL por sí sola no describe todo el endpoint: también importa el método

Qué hace un middleware

Un middleware se ejecuta dentro del recorrido de una solicitud. Puede registrar información, procesar JSON, comprobar un token, validar datos o detener la petición. Su orden importa: no puedes validar un cuerpo que aún no se ha procesado.

Nuestro logger imprime SE HA REALIZADO UNA SOLICITUD para las solicitudes del servidor. También añade un encabezado y registra método, ruta y código de respuesta para la sesión del laboratorio.

El botón Ver registro de solicitudes muestra ese trabajo en la web. No guarda contraseñas, cuerpos de login ni tokens. La función next continúa el recorrido; un error pasa al manejador final.

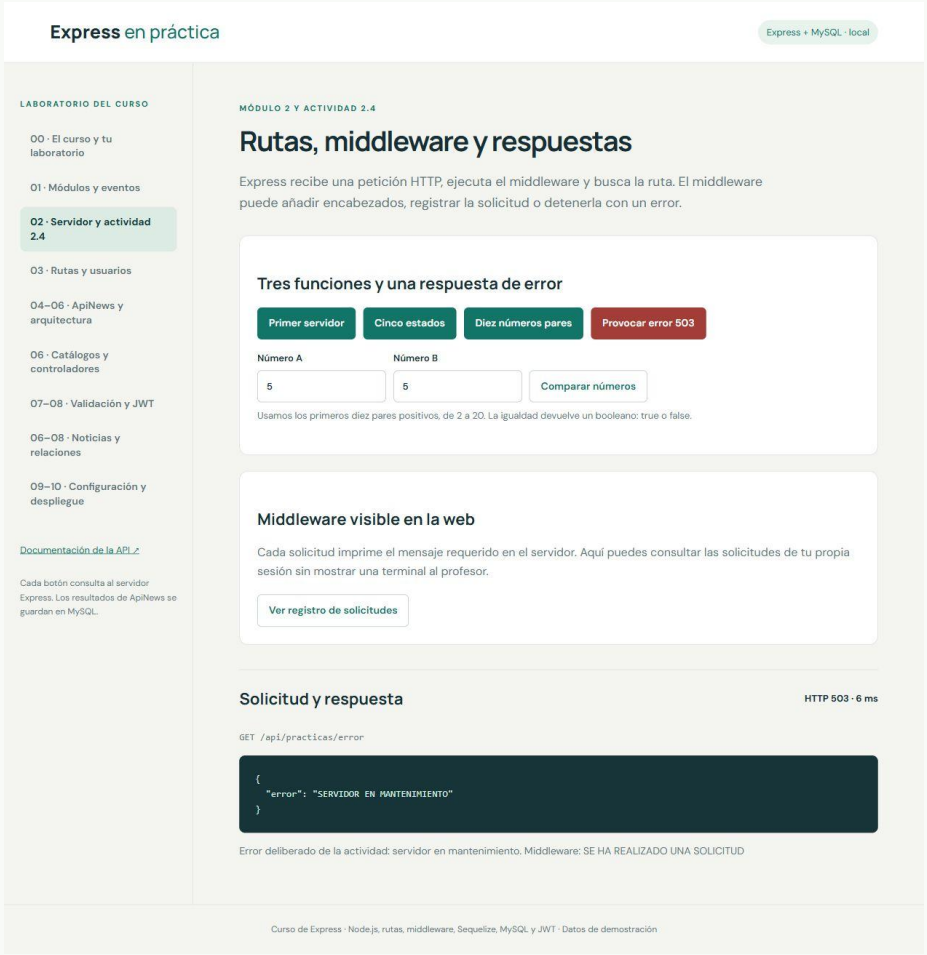
Las tres funciones de la actividad 2.4

Función	Entrada	Resultado esperado
Estados	Sin parámetros	Un arreglo de cinco estados de México
Pares	Sin parámetros	Diez pares positivos generados con for: 2 a 20
Igualdad	Dos números recibidos por la ruta	Un booleano: true si son iguales y false si no

Los parámetros de una URL llegan como texto. Los convertimos a números antes de compararlos y rechazamos entradas que no sean numéricas.

La actividad no aclara si la secuencia incluye cero. En este trabajo usamos los primeros diez pares positivos y lo indicamos en la demostración.

Un error 503 también es una actividad correcta



Provocar error 503 activa el caso solicitado por el curso: SERVIDOR EN MANTENIMIENTO. El manejador central responde con ese mensaje y el código 503.

Es un fallo deliberado de esa solicitud. El proceso sigue funcionando: después puedes consultar los estados o verificar la base de datos.

No confundas el error de práctica con una aplicación caída. Para demostrarlo, señala la ruta, el código HTTP y el mensaje recibido.

Métodos HTTP y operaciones

Método	Qué hacemos	Ejemplo del proyecto
GET	Consultar	Listar categorías o noticias
POST	Crear o enviar datos	Registrar usuario o proponer noticia
PUT	Actualizar los campos del registro	Editar una categoría o noticia
PATCH	Modificar una parte concreta	Aprobar o rechazar una noticia
DELETE	Eliminar	Borrar un usuario de práctica

El servidor puede utilizar la misma ruta con métodos diferentes. La autorización y la validación se comprueban para la operación concreta que se solicita.

Tres formas de enviar información

Parámetro de ruta: /api/teoria/usuarios/7 coloca el ID dentro del camino. Express lo recibe en req.params. Es útil cuando buscas un recurso determinado.

Parámetro de consulta: /api/teoria/busqueda?nombre=Ana&edad=22 añade filtros después de ?. Express los recibe en req.query. La ruta base sigue siendo la misma.

Cuerpo: una solicitud POST envía un objeto JSON. express.json lo procesa y el controlador lo recibe en req.body. En la web hay tres botones para comparar estos orígenes y revisar qué recibió realmente el servidor.

Usuarios en un arreglo de memoria

PracticaInformacionRutas empieza con cinco usuarios que tienen id, nombre y edad. La página permite listar, filtrar por nombre o edad mínima, agregar con POST, consultar por ID y eliminar con DELETE.

Al agregar un usuario se asigna un ID nuevo. Eliminarlo no obliga a renumerar los demás. Si consultas un ID que ya no existe, recibes 404. Entradas como una edad negativa producen 422.

Estos datos viven en memoria y se reinician al reiniciar el servidor. Cada sesión de navegador tiene un arreglo independiente. Esta práctica enseña rutas; los usuarios de ApiNews, en cambio, se guardan en MySQL.

Qué resuelve ApiNews

ApiNews organiza noticias por categorías y estados de México. Una persona se registra como contribuidora, inicia sesión y propone una noticia. Un administrador revisa el contenido y decide si se publica.

El proyecto utiliza cinco tablas: perfiles, usuarios, estados, categorías y noticias. Cada noticia identifica su categoría, estado y autor. El usuario identifica su perfil y estado.

Los botones del laboratorio demuestran consultas, altas, modificaciones y bajas. La información persistente se guarda en MySQL. Reiniciar Express no elimina esas filas, a diferencia del arreglo de la práctica inicial.

Las carpetas y su responsabilidad

Ubicación	Responsabilidad
app.js y config.js	Iniciar Express y leer la configuración del entorno
routes/ y controllers/	Elegir la operación y ejecutar su lógica
models/ y config.db.js	Definir tablas, asociaciones y conexión MySQL
validators/ y middlewares/	Revisar datos, registrar solicitudes y comprobar acceso
public/ y practicas/	Interfaz de demostración y ejercicios independientes
docs/ y tests/	Documentación, solicitudes y pruebas de integración

Para localizar una actividad, empieza en su ruta. Sigue el controlador y revisa el modelo que utiliza. Si la pantalla cambia, busca su sección en public/app.js.

El recorrido de una consulta



LABORATORIO DEL CURSO

00 · El curso y tu laboratorio

01 · Módulos y eventos

02 · Servidor y actividad 2.4

03 · Rutas y usuarios

04–06 · ApiNews y arquitectura

06 · Catálogos y controladores

07–08 · Validación y JWT

06–08 · Noticias y relaciones

09–10 · Configuración y despliegue

Documentación de la API ↗

MÓDULOS 4, 5 Y 6

La arquitectura de ApiNews

Los archivos se organizan según su responsabilidad. Las rutas deciden a quién llamar; los controladores aplican la operación; los modelos representan las tablas y sus relaciones.

Solicitud HTTP → Middleware → Ruta y validación → Controlador → Modelo y MySQL

Carpeta o archivo	Responsabilidad
app.js	Configura Express y conecta todos los bloques.
routes/	Relaciona cada método y URL con la operación que corresponde.
controllers/	Consulta, crea, actualiza y elimina registros.
models/	Define perfiles, usuarios, estados, categorías, noticias y sus relaciones con Sequelize.
validators/	Revisa la entrada antes de guardar.
middlewares/	Registra solicitudes y comprueba JWT y permisos.
config.db.js y .env	Conexión a MySQL y configuración privada.
public/	La página que estás utilizando para demostrar las prácticas.

El navegador pide GET /api/noticias. La ruta llama a NewController. El controlador consulta el modelo New y solicita sus relaciones con include.

Sequelize genera SQL para MySQL. La base devuelve los registros; el servidor envía JSON y la página lo presenta.

La imagen muestra la arquitectura de nuestra aplicación. Puedes seguir ese mismo recorrido en los archivos cuando quieras entender o cambiar una operación.

Relaciones entre los modelos

Modelo	Se relaciona con	Qué recuperas al consultar
Usuario	Perfil y estado	Rol del usuario y estado al que pertenece
Usuario	Noticias	Noticias creadas por esa persona
Noticia	Categoría	Tema de la noticia
Noticia	Estado	Lugar al que corresponde
Noticia	Autor	Nombre del usuario que la creó

Las claves foráneas relacionan los IDs. include permite devolver la información relacionada, no solo el número. MySQL impide eliminar un registro que sigue siendo necesario para otras filas.

Sequelize y los controladores

Sequelize es el ORM: trabajas con modelos de JavaScript y la biblioteca genera consultas SQL. findAll lista; findByPk busca un ID; create inserta; update modifica y destroy elimina.

Cada controlador usa estas operaciones para su recurso. Creamos controladores y rutas de perfiles, categorías, estados, usuarios y noticias. Los catálogos comparten una factoría CRUD para evitar repetir la misma lógica.

Las rutas conectan método, camino, validación y controlador. El controlador no debe confiar en que el formulario ya revisó los datos. La base también aplica restricciones como correos únicos y relaciones válidas.

Validaciones antes de guardar

Express Validator revisa lo que llega al servidor. En el registro verifica nombres, correo, contraseña y estado. En las noticias verifica título, contenido y referencias a categorías y estados.

Si una regla falla, validationResult permite construir una respuesta 422 con el campo y el mensaje. El laboratorio tiene un botón que envía datos inválidos directamente para demostrar que la protección no depende del formulario.

Un correo repetido produce 409 porque ya existe. Un ID relacionado inexistente también genera un conflicto. Validar formato y conservar integridad en la base son controles complementarios, y se realizan antes de dar por correcta una operación.

Registro y contraseñas

La persona envía nombre, apellido, correo, contraseña y estado. El servidor valida la entrada, asigna el perfil Contribuidor y guarda un hash de la contraseña con bcrypt. La respuesta no incluye el hash.

No guardamos una contraseña como texto en la tabla. En el login, bcrypt compara el valor recibido con el hash almacenado. No se recupera la contraseña original desde la base.

El cliente no puede convertirse en administrador enviando perfil_id o estatus en el formulario. Esas decisiones las toma el servidor. La cuenta administradora de demostración se prepara mediante configuración privada.

JWT y la sesión

Al iniciar sesión correctamente, ApiNews devuelve un JSON Web Token firmado. El cliente lo envía como Authorization: Bearer seguido del token cuando consulta una ruta protegida.

El middleware verifica la firma, vencimiento, emisor y destinatario. Luego consulta al usuario en la base para comprobar que exista y esté activo. Sin token, o con uno falso o vencido, responde 401.

El JWT está firmado; no es una contraseña ni un documento cifrado. En nuestro laboratorio se mantiene en memoria. Cerrar sesión o recargar elimina el token del navegador. La clave de firma solo existe en el entorno privado del servidor.

Autenticación y autorización

Acción	Acceso	Respuesta si falta acceso
Consultar noticias aprobadas	Público	No requiere token
Proponer una noticia	Usuario autenticado	401 sin sesión
Editar una noticia	Autor o administrador	403 si pertenece a otra persona
Moderar noticias	Administrador	403 para un contribuidor
Modificar catálogos o gestionar usuarios	Administrador	401 sin sesión o 403 sin permiso

Autenticación responde quién eres. Autorización responde qué puedes hacer. Por eso una sesión válida no da permiso para todas las operaciones.

El flujo de una noticia

Crear → pendiente. El usuario envía la noticia, pero la API la guarda sin publicarla. El autor la puede ver en Ver mis noticias; un administrador puede ver las pendientes para moderar.

Aprobar → visible. El administrador utiliza PATCH para cambiar el estatus a aprobada. Ahora la consulta pública GET /api/noticias la devuelve junto con categoría, estado y autor. Rechazar la mantiene fuera de la lista pública.

Editar → pendiente de nuevo. Aunque la noticia estuviera aprobada, modificarla requiere otra revisión. El servidor toma el autor del usuario autenticado y no acepta que el cliente suplante a otra persona.

CORS JSON y configuración privada

CORS indica desde qué orígenes puede un navegador consultar la API. En este laboratorio los clientes pueden leer la API pública; las operaciones protegidas siguen exigiendo un token y el rol correspondiente. CORS no sustituye la autorización.

express.json acepta cuerpos de hasta 2 MB. El campo opcional imagen puede transportar texto base64 o una URL; no se ejecuta ni se inserta como HTML. Un JSON mal formado devuelve 400 y uno demasiado grande devuelve 413.

dotenv carga .env en desarrollo. Render proporciona las variables del alojamiento: conexión MySQL, clave JWT y contraseña administradora. Los archivos privados quedan excluidos de GitHub.

Documentación y pruebas

Swagger se abre en /docs y describe 47 operaciones: métodos, rutas, parámetros, cuerpos y autenticación. La colección Postman permite repetir las solicitudes; el login guarda el JWT en una variable de la colección.

Se realizaron 14 pruebas de integración con una base MySQL temporal independiente. Comprueban funciones, errores, filtros, altas y bajas, validaciones, JWT, permisos, relaciones y aprobación de noticias. La base de demostración no se borra.

También se probó la interfaz en el navegador: creación y aprobación de una noticia, consultas y sesión. Para estudiar, mira qué entrada envías, el código HTTP recibido y el cambio que aparece en la base o en la lista.

Dónde se aloja el trabajo

GitHub guarda el código en https://github.com/edcruzek/ApiNews. Render ejecuta el servidor Express y sirve la página. Clever Cloud aloja una base MySQL independiente en el plan DEV.

El enlace de la demostración es https://api-news-edcruzek.onrender.com. El mismo origen sirve la interfaz, /api y /docs. No tienes que iniciar VS Code ni una terminal para presentarlo cuando el servicio está activo.

Se eligieron planes gratuitos. Render puede suspender la instancia por inactividad y tardar en reactivarla. Clever Cloud DEV tiene 10 MB y cinco conexiones: es una base pequeña para esta práctica, no un servicio de noticias comercial.

Cómo presentar las actividades al profesor

1. Abre el enlace y verifica Express con MySQL. 2. Calcula operaciones y emite un saludo. 3. Muestra los estados, pares, igualdad, error 503 y registro visible de solicitudes.

4. En Usuarios de práctica, filtra, agrega, consulta y elimina. 5. En JWT, envía datos inválidos, registra un contribuidor e inicia sesión. 6. Crea una noticia y observa que queda pendiente.

7. Entra con la cuenta administradora, consulta catálogos y usuarios, aprueba la noticia y revisa que aparezca públicamente. 8. Abre Swagger para mostrar la documentación. Explica siempre qué dato entra, qué hace Express y qué respuesta sale.

Cómo interpretar las respuestas

Código	Qué significa en la práctica
200 o 201	Consulta correcta o registro creado
204	Se eliminó el registro y no se envía cuerpo
401 y 403	Falta sesión válida o falta permiso
404 y 409	No existe el registro o entra en conflicto con otros datos
422	Los datos no cumplen las reglas del servidor
503	El ejemplo intencional de mantenimiento

Si aparece un error de conexión, primero verifica salud. Si los datos son rechazados, lee detalles. Si falla el acceso, revisa el login y el perfil. Una respuesta de error esperada también demuestra que la API funciona correctamente.

Lo que debes poder explicar

Un endpoint combina método y ruta. Un middleware participa en el recorrido. Un controlador ejecuta la operación. Un modelo describe una tabla y sus relaciones. JSON es el formato de los mensajes entre cliente y servidor.

Sequelize consulta MySQL; MySQL conserva los datos. La validación rechaza entradas incorrectas. El JWT identifica una sesión; el rol limita lo que esa sesión puede hacer. La aprobación decide qué noticias se vuelven públicas.

No necesitas memorizar todo el código. Practica una solicitud de principio a fin y explica estos vínculos con tus palabras. El laboratorio y los ejemplos del informe están preparados para que puedas comprobar cada idea.